



Data Visualization

LABORATORIO

Capitolo 2 – Introduzione JavaScript



Argomenti del giorno

- **Concetti base di JavaScript**
 - Tipi, dichiarazione variabili
 - Operatori aritmetici
 - Operatori logici
 - Costrutti di controllo
 - Costrutti di iterazione
 - Funzioni callback

Dichiarazione variabili

Una variabile in JavaScript può essere dichiarata tramite le keyword **let** e **const**.

- **let** permette di creare delle variabili a cui è possibile riassegnare dei valori.
- **const** invece, come dice il nome, permette di creare una costante, quindi una variabile a cui non è possibile riassegnare dei valori.

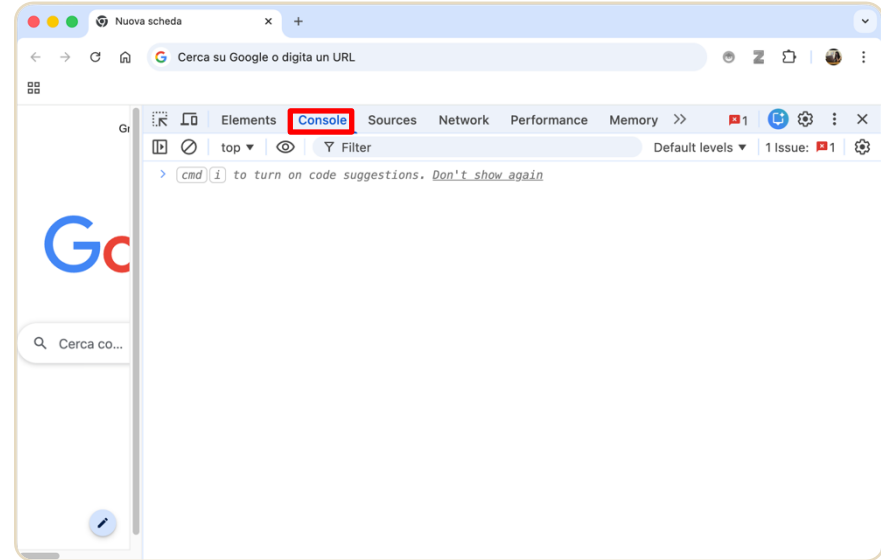
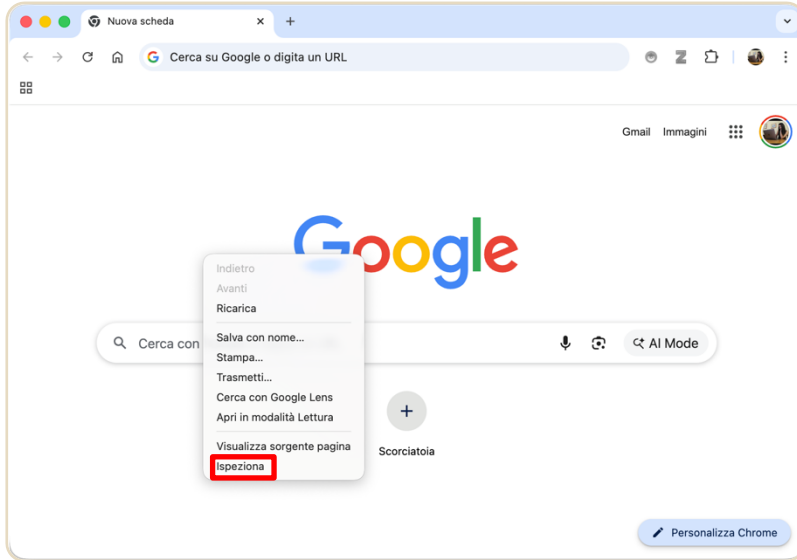
Tip: esiste anche la keyword *var*, che veniva utilizzata in passato al posto di **let**. Oggi è **obsoleta** e **meglio non utilizzarla!**

Con il tag HTML **<script>** possiamo incorporare codice eseguibile. Noi lo useremo per collegare il codice JS alla nostra pagina HTML

```
<head>
  <title>Title</title>
  <meta charset="UTF-8">
  <link rel="stylesheet" href="css/style.css">
  <script type="text/javascript" src="js/main.js"></script>
</head>
```

Console Javascript sul browser

- Click destro > **Ispeziona** (oppure **Ispeziona elemento** oppure **Analizza**)



Tipi di dato primitivi

In **JavaScript**, le variabili non vengono dichiarate di un determinato tipo, ma assumono quello del valore che viene loro assegnato. Esistono due categorie di tipi, la prima è quella dei **tipi primitivi**:

- **Number**: in JS non c'è distinzione tra *int*, *float*, *double*, etc.. Sono tutti *number*
- **String**: una sequenza di caratteri
- **Boolean**: rappresenta *true* o *false*
- **Null**: indica l'assenza di valore
- **Undefined**: indica una variabile dichiarata ma non *inizializzata*

```
js main.js x
1 console.log("Ciao!!!")
2
3 let x : number = 5;
4 console.log ("\\nDichiaro la variabile x, il suo valore è: ", x);
5
6 x = 6;
7 console.log ("\\nRiassegno un nuovo valore alla variabile x, ora è: ", x);
8
9 const y : number = 10;
10 console.log ("\\nDichiaro la costante y, il suo valore è: ", y);
11 //y = 11; // Questo causerà un errore, perché non possiamo riassegnare un nuovo valore a una costante
12
13 x = "Hello";
14 console.log ("\\nx = \\\"Hello\\\", x : ", x);
15
16 x = true;
17 console.log ("\\nx = true, x : ", x);
18
19 let z;
20 console.log ("\\nDichiaro la variabile z senza assegnarle un valore, z : ", z);
21
22 z = null;
23 console.log ("\\nAssegno a z il valore null, z : ", z);
```

Tipi di dato di riferimento

I **tipi di riferimento** differiscono dai tipi primitivi in quanto memorizzano i dati come un riferimento in memoria e hanno una struttura ben diversa.

- **Array**: una collezione di valori dello stesso tipo. Benché venga riconosciuto in questo modo dall'IDE, il suo tipo sarà comunque **Object**
- **Object**: keyword per rappresentare una generica collezione di proprietà e valori

Altri tipi di riferimento: **Date**, **Map**, **Set**, **RegExp**, etc...

```
main.js x
49 function dataTypes(): void { Show usages
112 // Array
113 let arr: number[] = [1, 2, 3, 4, 5];
114 console.log(`\nlet arr = [1, 2, 3, 4, 5] ==> arr is an ${typeof arr}`);
115 // This will print "object" to the console.
116
117 // Object (it's like a dictionary in Python, or a map in Java)
118 let obj: {...} = {
119     name: "John",
120     age: 30,
121     "salary": 1234.56,
122     "isMarried": true,
123     "address": {street: "123 Main St", city: "New York"},
124     childrenAges: [5, 7, 9]
125 };
126 console.log(`\nlet obj = {
127     name: "John",
128     age: 30,
129     "salary": 1234.56,
130     "isMarried": true,
131     "address": {street: "123 Main St", city: "New York"},
132     childrenAges: [5, 7, 9]
133 }; ==> obj is an ${typeof obj}`); // This will print "object" to the console.
134 }
```

Tipi di dato di riferimento

Anche le **funzioni** rientrano nei tipi di riferimento, questo perché una funzione non è altro che un **oggetto** che può essere **invocato**. Ciononostante, il suo tipo è **function**.

Esistono diversi modi per dichiarare una funzione:

- Assegnare a una variabile una funzione tramite la keyword **function**
- Utilizzare direttamente la keyword **function** e assegnarle un **nome**
- Assegnare a una variabile una **arrow function**. Se quest'ultima non è assegnata a una variabile, non sarà riutilizzabile perché non ne avremo il riferimento.

```
let func = function() : void { Show usages
  console.log("Ciao, sono una funzione!");
}

function func2() : void { Show usages
  console.log("Ciao, sono un'altra funzione!");
}

let func3 = () : void => { Show usages
  console.log("Ciao, sono una arrow function!");
}

console.log("\nRichiamo le funzioni: ");
func()
func2()
func3()
```

Operatori aritmetici

Gli operatori **aritmetici** sono gli stessi che troviamo in praticamente tutti i linguaggi di programmazione **moderni**.

Stesso discorso per l'operatore di assegnamento e quelli di assegnamento combinato.

```
main.js x
function operators():void { Show usages
  // Arithmetic operators
  let x : number = 5;
  let y : number = 2;
  console.log(`\nlet x = 5; let y = 2; ==> x + y = ${x + y}`); // This will print "7" to the console.
  console.log(`x - y = ${x - y}`); // This will print "3" to the console.
  console.log(`x * y = ${x * y}`); // This will print "10" to the console.
  console.log(`x / y = ${x / y}`); // This will print "2.5" to the console.
  console.log(`x % y = ${x % y}`); // This will print "1" to the console.
  console.log(`x ** y = ${x ** y}`); // This will print "25" to the console.

  // Assignment operators
  x = 5;
  console.log(`\nlet x = 5; ==> x = ${x}`); // This will print "5" to the console.
  x += 3;
  console.log(`x += 3; ==> x = ${x}`); // This will print "8" to the console.
  x -= 3;
  console.log(`x -= 3; ==> x = ${x}`); // This will print "5" to the console.
  x *= 3;
  console.log(`x *= 3; ==> x = ${x}`); // This will print "15" to the console.
  x /= 3;
  console.log(`x /= 3; ==> x = ${x}`); // This will print "5" to the console.
  x %= 3;
  console.log(`x %= 3; ==> x = ${x}`); // This will print "2" to the console.
  x **= 3;
  console.log(`x **= 3; ==> x = ${x}`); // This will print "8" to the console.
}
```


Operatori di confronto

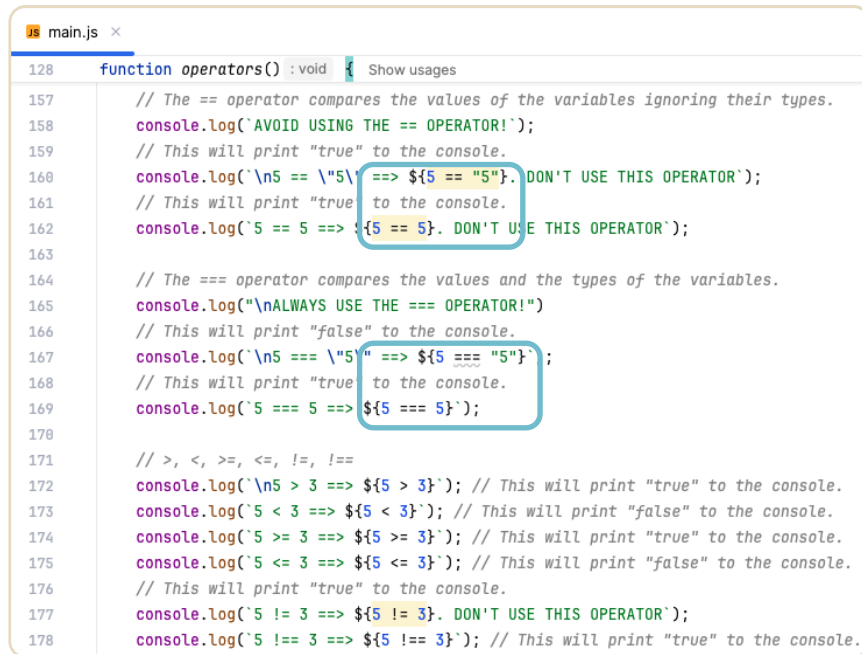
Gli operatori `==` e `===` sono operatori di uguaglianza, ma si comportano in modo **diverso** in base a come confrontano i valori.

Uguaglianza debole (`==`)

- Confronta i valori dopo aver applicato una conversione automatica.
- Se i due valori non sono dello stesso tipo, JavaScript tenta di **convertirli** in un tipo **comune** prima di confrontarli.

Uguaglianza rigorosa (`===`)

- Confronta sia il valore che il tipo dei due operandi.
- Non effettua alcuna conversione automatica. I due valori devono essere **identici** sia nel tipo che nel valore per risultare **true**.



```
128 function operators() : void { Show usages
157 // The == operator compares the values of the variables ignoring their types.
158 console.log("AVOID USING THE == OPERATOR!");
159 // This will print "true" to the console.
160 console.log(`\n5 == \5` ==> ${5 == "5"}. DON'T USE THIS OPERATOR`);
161 // This will print "true" to the console.
162 console.log(`5 == 5 ==> ${5 == 5}. DON'T USE THIS OPERATOR`);
163
164 // The === operator compares the values and the types of the variables.
165 console.log("\nALWAYS USE THE === OPERATOR!")
166 // This will print "false" to the console.
167 console.log(`\n5 === \5` ==> ${5 === "5"} `);
168 // This will print "true" to the console.
169 console.log(`5 === 5 ==> ${5 === 5}`);
170
171 // >, <, >=, <=, !=, !==
172 console.log(`\n5 > 3 ==> ${5 > 3}`); // This will print "true" to the console.
173 console.log(`5 < 3 ==> ${5 < 3}`); // This will print "false" to the console.
174 console.log(`5 >= 3 ==> ${5 >= 3}`); // This will print "true" to the console.
175 console.log(`5 <= 3 ==> ${5 <= 3}`); // This will print "false" to the console.
176 // This will print "true" to the console.
177 console.log(`5 != 3 ==> ${5 != 3}. DON'T USE THIS OPERATOR`);
178 console.log(`5 !== 3 ==> ${5 !== 3}`); // This will print "true" to the console.
```

Operatori di confronto

Gli operatori `<`, `>`, `<=`, `>=`, come per gli operatori aritmetici, sono gli stessi che troviamo in altri linguaggi di programmazione.

Esistono inoltre gli operatori di disuguaglianza `!=` e `!==`, corrispettivi opposti di quelli di uguaglianza, infatti la logica di funzionamento è la medesima.

```
main.js x
128 function operators() : void { Show usages
157 // The == operator compares the values of the variables ignoring their types.
158 console.log('AVOID USING THE == OPERATOR!');
159 // This will print "true" to the console.
160 console.log(`\n5 == "5" ==> ${5 == "5"}. DON'T USE THIS OPERATOR`);
161 // This will print "true" to the console.
162 console.log(`5 == 5 ==> ${5 == 5}. DON'T USE THIS OPERATOR`);
163
164 // The === operator compares the values and the types of the variables.
165 console.log(`\nALWAYS USE THE === OPERATOR!`)
166 // This will print "false" to the console.
167 console.log(`\n5 === "5" ==> ${5 === "5"}`);
168 // This will print "true" to the console.
169 console.log(`5 === 5 ==> ${5 === 5}`);
170
171 // >, <, >=, <=, !=, !==
172 console.log(`\n5 > 3 ==> ${5 > 3}`); // This will print "true" to the console.
173 console.log(`5 < 3 ==> ${5 < 3}`); // This will print "false" to the console.
174 console.log(`5 >= 3 ==> ${5 >= 3}`); // This will print "true" to the console.
175 console.log(`5 <= 3 ==> ${5 <= 3}`); // This will print "false" to the console.
176 // This will print "true" to the console.
177 console.log(`5 != 3 ==> ${5 != 3}. DON'T USE THIS OPERATOR`);
178 console.log(`5 !== 3 ==> ${5 !== 3}`); // This will print "true" to the console.
```

Operatori logici

Anche gli operatori logici di **&& (AND)**, **|| (OR)** e **! (NOT)** sono i medesimi degli altri linguaggi di programmazione.

Ricordiamo che **AND** e **OR** sono **corto-circuitati**:

- **AND**: se il primo valore del confronto è **false**, il secondo non viene nemmeno valutato e **restituisce false**.
- **OR**: viceversa, se il primo valore del confronto è **true**, il secondo non viene valutato e **restituisce true**.

```
js main.js x
128 function operators() : void { Show usages
129
180 // Logical operators
181 // &&, ||, !
182 // This will print "true" to the console.
183 console.log(`\ntrue && (and) true ==> ${true && true}`);
184 // This will print "false" to the console.
185 console.log(`true && (and) false ==> ${true && false}`);
186 // This will print "true" to the console.
187 console.log(`true || (or) true ==> ${true || true}`);
188 // This will print "true" to the console.
189 console.log(`true || (or) false ==> ${true || false}`);
190 // This will print "false" to the console.
191 console.log(`false || (or) false ==> ${false || false}`);
192
193 // The ! operator negates the value of a boolean variable.
194 console.log(`!true ==> ${!true}`); // This will print "false" to the console.
195 console.log(`!false ==> ${!false}`); // This will print "true" to the console.
196 }
```

Costrutti di controllo

Anche i costrutti di controllo sono i medesimi degli altri linguaggi di programmazione:

- **If**: esegue un blocco di codice solo se la condizione è **true**.
- **If...else**: se la condizione de l'**if** è falsa, allora esegue il blocco di codice **dell'else**.
- **If...else if...else**: una **concatenazione** di if-else, utile quando ci sono **più condizioni** da controllare.

```
main.js
198 function controlStructures(): void { Show usages
199     // If statement
200     let x : number = 5;
201     if (x > 3) {
202         // This will print "x is greater than 3" to the console.
203         console.log(`\nif (x > 3) {console.log("x is greater than 3")}`);
204     }
205
206     // If-else statement
207     if (x > 3) {
208         // This will print "x is greater than 3" to the console.
209         console.log(`\nif (x > 3) {console.log("x is greater than 3")}`);
210         else {console.log("x is less than or equal to 3")}`);
211     } else {
212         console.log("x is less than or equal to 3");
213     }
214
215     // If-else if-else statement
216     if (x > 3) {
217         // This will print "x is greater than 3" to the console.
218         console.log(`\nif (x > 3) {console.log("x is greater than 3")}`);
219     } else if (x < 3) {
220         console.log("x is less than 3");
221     } else if (x === 3) {
222         console.log("x is equal to 3");
223     } else {
224         console.log("x is not a number");
225     }
```

Costrutti di controllo

Anche il costrutto **switch** è il medesimo. È utile quando ci sono molte condizioni che confrontano un valore con più possibili casi.

Ricordiamoci che **senza break**, l'esecuzione **continua** nei case successivi fino alla fine o finché non si trova un break.

È buona norma aggiungere sempre un case **default**.

```
main.js
function controlStructures(): void { Show usages
235 // can be converted in a switch statement as follows:
236 let day : string = "Monday";
237 switch (day) {
238     case "Monday":
239         console.log("Today is Monday");
240         break;
241     case "Tuesday":
242         console.log("Today is Tuesday");
243         break;
244     case "Wednesday":
245         console.log("Today is Wednesday");
246         break;
247     case "Thursday":
248         console.log("Today is Thursday");
249         break;
250     case "Friday":
251         console.log("Today is Friday");
252         break;
253     case "Saturday":
254         console.log("Today is Saturday");
255         break;
256     case "Sunday":
257         console.log("Today is Sunday");
258         break;
259     default:
260         console.log("Today is not a day of the week");
261 }
```

Costrutti di iterazione

Il ciclo **for** è utile quando si conosce il numero esatto di iterazioni da eseguire.

In JavaScript, e in altri linguaggi moderni, esistono le varianti **for...in** usata per iterare su **oggetti** e **for...of** per iterare su array e **stringhe**.

Il ciclo **while** è utile quando non si sa in anticipo quante volte ripetersi.

La variante **do...while** esegue almeno una iterazione poiché la condizione da valutare viene dopo.

```
main.js ×
function controlStructures() : void { Show usages
198
263 // For loop
264 console.log("\nfor (let i = 0; i < 5; i+=2) {console.log(i)}");
265 for (let i : number = 0; i < 5; i += 2) {
266     console.log(i);
267 }
268 // end for loop
269
270 // While loop
271 console.log("\nlet i = 0; while (i < 5) {console.log(i); i++}");
272 let i : number = 0;
273 while (i < 5) {
274     console.log(i);
275     i++;
276 }
277 }
```

```
107 // for ... in ...
108 const myObj : { ... } = { a: 1, b: 2, c: 3 };
109 for (const key in myObj) {
110     console.log(`${key}: ${myObj[key]}`);
111 }
112
113 // for ... of ...
114 const array1 : number[] = [10, 20, 30];
115 for (const element : number of array1) {
116     console.log(element);
117 }
```

Funzioni callback

```
function scaricaDati(){ Show usages
  let datiScaricati;

  console.log("1. Avvio download dei dati...");

  setTimeout( handler: () : void => {
    datiScaricati = {utente: "Marco", ruolo:"Admin"};
    console.log("--> (Dietro le quinte) Download completato!");
  }, timeout: 2000);

  return datiScaricati;
}

function mostraDati(dati) : void { Show usages
  console.log("2. I dati ottenuti sono: ", dati);
  console.log("3. Mostro i dati nella pagina web.");
}

let risultato = scaricaDati();
mostraDati(risultato);
```

In console...

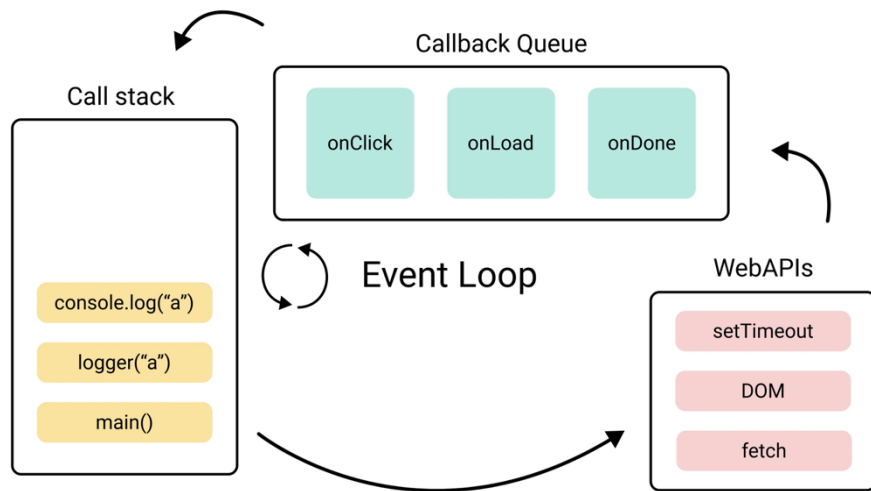
1. Avvio download dei dati...
 2. I dati ottenuti sono: `undefined`
 3. Mostro i dati nella pagina web.
- > (Dietro le quinte) Download completato!

Ma come? Ci aspettavamo che l'ordine fosse:

1. Avvio download dei dati...
→ (Dietro le quinte) Download completato!
2. I dati ottenuti sono: "utente: Marco, ruolo: Admin"
3. Mostro i dati nella pagina web

Perché non ha aspettato che finisse?

Perché le callback sono necessarie?



JavaScript è single-threaded quindi non va in parallelo.
Immaginatevi se si dovesse bloccare per un'azione... si congelerebbe tutta la pagina web!

Ci sono tre componenti fondamentali:

- **Call Stack:** JS esegue riga per riga tutto ciò che è presente nello stack (LIFO)
- **Web Apis:** browser esegue le azioni bloccanti liberando lo stack.
- **Callback(task) queue:** risiedono le funzioni callback ricevute dalle web api. Nel mentre, l'Event Loop monitora costantemente il Call Stack: non appena risulta vuoto, preleva la prima callback in e la sposta nel Call Stack per farla eseguire

Funzioni callback

Una **callback** è una funzione passata come argomento a un'altra funzione, che verrà poi eseguita in un momento successivo.

Le callback servono per:

- Eseguire codice in modo **asincrono** (es. leggere un file, fare una richiesta HTTP)
- **Personalizzare** il comportamento di una funzione
- **Modulare** e riutilizzare codice

In console...

1. Avvio il download dei dati...

Nel frattempo javascript continua a eseguire...

—> (Dietro le quinte) Download completato!

2. I dati ottenuti sono: ▶ {utente: 'Marco', ruolo: 'Admin'}

3. Mostro i dati nella pagina web.

```
// Funzione che simula il download di dati
function scaricaDati(callback) : void { Show usages
  let datiScaricati;

  console.log("1. Avvio il download dei dati...");

  // Simuliamo l'attesa del server (2 secondi)
  setTimeout( handler: () : void => {
    datiScaricati = { utente: "Marco", ruolo: "Admin" };
    console.log("---> (Dietro le quinte) Download completato!");
    callback(datiScaricati)
  }, timeout: 2000);
}

let callbackMostraDati = function (risultato) : void { Show usages
  console.log("2. I dati ottenuti sono:", risultato);
  console.log("3. Mostro i dati nella pagina web.");
}

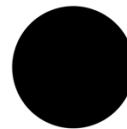
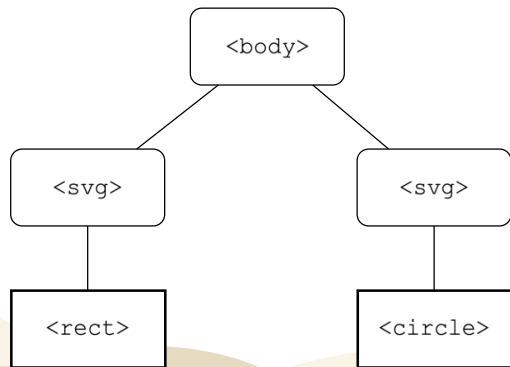
scaricaDati(callbackMostraDati);
console.log("Nel frattempo javascript continua a eseguire...");
```

DOM

Il **DOM (Document Object Model)** è una rappresentazione ad **albero** della pagina HTML. JavaScript può manipolarlo per:

- **Selezionare** elementi
- **Modificare** contenuti e attributi
- **Aggiungere o rimuovere** elementi

DOM della pagina HTML



```
<body>
<svg width="300" height="200">
  <rect x="125" y="75" width="100" height="50" >/rect>
</svg>

<svg width="300" height="200">
  <circle r="50" cx="150" cy="100">/circle>
</svg>

</body>
</html>
```

Selezionare con il DOM

Si utilizza il DOM tramite la keyword **document** che rappresenta la pagina HTML.

- **querySelector()**: seleziona il **primo elemento** che corrisponde a un **selettore CSS**, che può essere un attributo, una classe, etc...
- **querySelectorAll()**: seleziona **tutti** gli elementi di un certo tipo o classe.
- **getElementById()**: seleziona l'elemento con quell'**univoco ID**.

N.B. Si può usare anche `querySelector()` per selezionare l'elemento con un certo ID

```
main.js x
349 function DOM() :void { Show usages
356 // Get the first text element and log it to the console.
357 const first_text : SVGTextElement = document.querySelector( selectors: "text");
358 console.log("The first text:", first_text);
359
360 // Get all the text elements and log them to the console.
361 const all_texts : NodeListOf<SVGTextElement> = document.querySelectorAll( selectors: "text");
362 console.log("All texts:", all_texts);
363
364 // Get the first element with the class "small" and log it to the console.
365 const first_classed_small : Element = document.querySelector( selectors: ".small");
366 console.log("The first .small:", first_classed_small);
367
368 // Get all the elements with the class "small" and log them to the console.
369 const all_classed_small : NodeListOf<Element> = document.querySelectorAll( selectors: ".small");
370 console.log("All .small:", all_classed_small);
371
372 // Get the first text contained in a group and log it to the console.
373 const first_text_inside_group : Element = document.querySelector( selectors: "g text");
374 console.log("first \"g text\":", first_text_inside_group);
375
376 // Get all the texts groups contained inside a group and log them to the console.
377 const all_text_inside_groups : NodeListOf<Element> = document.querySelectorAll( selectors: "g text");
378 console.log("all \"g text\":", all_text_inside_groups);
379
380 // Change the text to "Hello World".
381 first_text.textContent = "Hello World. I\\\"m not \\\"My\\\" anymore";
```

Creare un elemento con il DOM

Per creare un elemento da aggiungere al DOM si usano le funzioni :

- **createElement()**: aggiunge elementi base di HTML
- **createElementNS()**: aggiunge elementi che hanno uno specifico **namespace (NS)**.
Utilizzeremo questa funzione per creare SVG e XML.

Se creassimo un elemento "line" tramite **createElement()** non funzionerebbe.

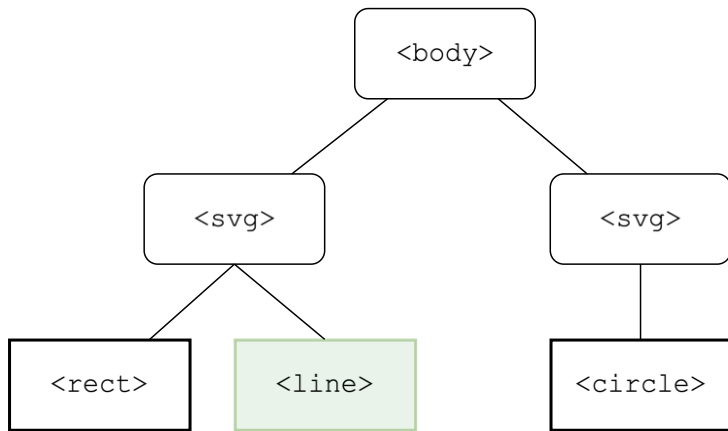
La funzione **setAttribute()** permette di aggiungere attributi all'elemento sul quale viene chiamata.

```
// È come se creassimo l'elemento nella pagina HTML così:  
// <line x1="60" y1="25" x2="60" y2="175" stroke="red"></line>
```

```
const svgNS : string = "http://www.w3.org/2000/svg"  
const line : SVGLineElement = document.createElementNS(svgNS, {qualifiedName: 'line'})  
line.setAttribute({qualifiedName: 'x1', value: '60'})  
line.setAttribute({qualifiedName: 'y1', value: '25'})  
line.setAttribute({qualifiedName: 'x2', value: '60'})  
line.setAttribute({qualifiedName: 'y2', value: '175'})  
line.setAttribute({qualifiedName: 'stroke', value: 'red'})  
// Lo inseriamo come figlio del primo svg che troviamo  
let svg : SVGSVGElement = document.querySelector({selectors: 'svg'})  
svg.appendChild(line)
```

Manipolazione del DOM

DOM della pagina HTML



```
// È come se creassimo l'elemento nella pagina HTML così:  
// <line x1="60" y1="25" x2="60" y2="175" stroke="red"></line>  
  
const svgNS : string = "http://www.w3.org/2000/svg"  
const line : SVGLineElement = document.createElementNS(svgNS, qualifiedName: 'line')  
line.setAttribute(qualifiedName: 'x1', value: '60')  
line.setAttribute(qualifiedName: 'y1', value: '25')  
line.setAttribute(qualifiedName: 'x2', value: '60')  
line.setAttribute(qualifiedName: 'y2', value: '175')  
line.setAttribute(qualifiedName: 'stroke', value: 'red')  
// Lo inseriamo come figlio del primo svg che troviamo  
let svg : SVGSVGElement = document.querySelector(selectors: 'svg')  
svg.appendChild(line)
```

Manipolazione del DOM

Un **event listener** è una funzione che viene eseguita quando si verifica un evento su un elemento del DOM (es. clic, passaggio del mouse, pressione di un tasto).

Tramite la funzione **addEventListener()** possiamo aggiungerne uno all'elemento sul quale si chiama la funzione.

```
let rect : Element = document.querySelector( selectors: 'svg rect')
rect.addEventListener( type: "mouseover", listener: () : void => {
  rect.setAttribute( qualifiedName: 'fill', value: 'blue');
})
rect.addEventListener( type: "mouseleave", listener: () : void => {
  rect.setAttribute( qualifiedName: "fill", value: "black");
})
```

Selezioniamo il primo rect figlio del primo svg nel DOM. Aggiungiamo due listener:

- Il primo gestisce il comportamento quando il mouse è sopra il rettangolo colorandolo di blue
- Il secondo gestisce il comportamento quando il mouse non è più sopra il rettangolo riportandolo al colore originale

Chaining functions

Le **chaining functions** (o "metodi concatenati") permettono di chiamare più metodi sulla stessa istanza, in una sola riga.

Queste funzioni particolari agiscono sullo stesso oggetto oppure passando l'output di una funzione come l'input dell'altra.

Si accede al DOM e si seleziona il primo svg tramite la funzione `querySelector('svg')` e a quell'elemento svg selezionato si inserisce nell'albero un figlio (quindi all'interno dell'SVG) in questo caso una linea precedentemente creata, la funzione è `appendChild(line)`.

```
// È come se creassimo l'elemento nella pagina HTML così:  
// <line x1="60" y1="25" x2="60" y2="175" stroke="red"></line>
```

```
const svgNS : string = "http://www.w3.org/2000/svg"  
const line : SVGLineElement = document.createElementNS(svgNS, {qualifiedName: 'line'})  
line.setAttribute({qualifiedName: 'x1', value: '60'})  
line.setAttribute({qualifiedName: 'y1', value: '25'})  
line.setAttribute({qualifiedName: 'x2', value: '60'})  
line.setAttribute({qualifiedName: 'y2', value: '175'})  
line.setAttribute({qualifiedName: 'stroke', value: 'red'})  
// Lo inseriamo come figlio del primo svg che troviamo  
let svg : SVGSVGElement = document.querySelector(selectors: 'svg').appendChild(line)  
  
let rect : Element = document.querySelector(selectors: 'svg rect')  
rect.addEventListener({type: "mouseover", listener: () : void => {  
  rect.setAttribute({qualifiedName: 'fill', value: 'blue'});  
}})  
rect.addEventListener({type: "mouseleave", listener: () : void => {  
  rect.setAttribute({qualifiedName: "fill", value: "black"};  
}})
```